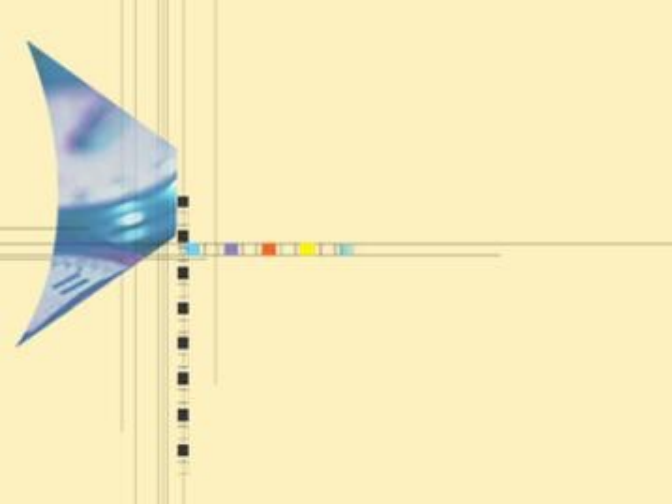


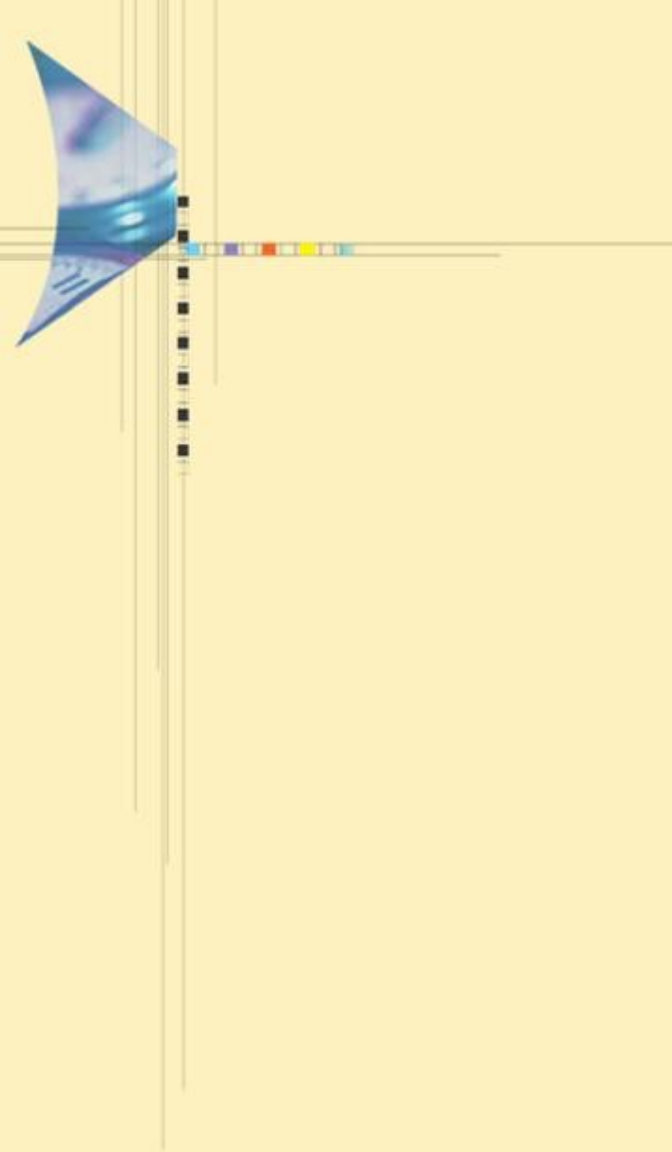


算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

周一(3-4，三区1-502)，周三（6-7，三区1-325）



第三讲

分 治

主要内容



3.1 引言

3.2 分治算法的简单例

3.3 分治策略的思想

3.4 分治策略的经典应用

3.5 分治算法的改进



主要内容



3.5. 分治算法的改进途径

3.5.1 减少子问题个数

3.5.2 减少递归内部计算量



回忆：主定理

主定理： 设 $a \geq 1, b > 1$ 为常数， $f(n)$ 为函数， $T(n)$ 为非负整数，且 $T(n) = aT(n/b) + f(n)$ ，则有以下结果：

1. 若 $f(n) = O(n^{\log_b a - \varepsilon})$, $\varepsilon > 0$, 那么 $T(n) = \Theta(n^{\log_b a})$
2. 若 $f(n) = \Theta(n^{\log_b a})$, 那么 $T(n) = \Theta(n^{\log_b a} \log n)$
3. 若 $f(n) = \Omega(n^{\log_b a + \varepsilon})$, $\varepsilon > 0$, 且对于某个常数 $c < 1$ 和充分大的 n 有 $a f(n/b) \leq c f(n)$, 那么 $T(n) = \Theta(f(n))$

- 与通常的蛮力算法相比，分治算法的时间复杂度确实有明显改进。
- 但是分治策略并不是处处有效的。对有些问题，简单的分治策略对提高效率并没有用处，其原因主要在于分治算法的递归调用：
 - (1) 产生的子问题个数较多。一般采用的二分法， $b=2$ ，如果一次调用3个以上的子问题 ($a>3$)，往往会有 $T(n)=O(n^{\log_b a})$ 。 a 越大，函数阶越高，时间复杂度越高。
 - (2) 递归过程内部的工作量过多。 $f(n)$ 的阶越高，时间复杂度越高。

改进途径： (1) 寻找子问题之间依赖关系，减少子问题个数；

(2) 算法设计时，尽量把某些工作放到递归之外，作为预处理，减少递归内部调用的工作量。

3.5.1 改进途径1-减少子问题个数

- 挖掘子问题之间关系，通过代数变换等方法，减少子问题个数。
 - 大整数乘法（位乘法）
 - 矩阵乘法



大整数乘法

问题描述

- 一般假定大小一定的整数相乘需要耗费定量的时间，而当两个任意长的整数相乘时，这个假定就不再有效了。
- 对于处理不定长数的算法的数据输入，通常是按二进制的位数来计算的(或按二进制数字来计算)
- 设 u 和 v 是两个 n 位的整数，传统的乘法算法需要 $\Theta(n^2)$ 数字相乘来计算 u 和 v 的乘积。

? Is there much more efficient method?

大整数乘法

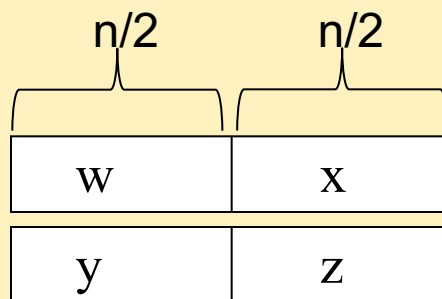
- 设 u 和 v 都是 n 位的二进制整数，现在要计算它们的乘积 $u*v$ 。
- 我们可以用小学所学的方法来设计一个计算乘积 $u*v$ 的算法，但是这样做计算步骤太多，显得效率较低。如果将每2个1位数的乘法或加法看作一步运算，那么这种方法要作 $O(n^2)$ 步运算才能求出乘积 $u*v$ 。
- 下面我们用分治法来设计一个大整数乘积算法。
 - 我们将 n 位的二进制整数 u 和 v 各分为2段，每段的长为 $n/2$ 位（为简单起见，假设 n 是2的幂）

大整数乘法

假定 n 是2的幂

$$u = w2^{n/2} + x:$$

$$v = y2^{n/2} + z:$$



u 和 v 的乘积可以计算为:

$$uv = (w2^{n/2} + x)(y2^{n/2} + z) = wy2^n + (xy + wz)2^{n/2} + xz$$

时间复杂度函数为:

$$T(n) = \begin{cases} d & \text{if } n = 1 \\ 4T(n/2) + bn & \text{if } n > 1 \end{cases}$$

递推式的解:

$$T(n) = \Theta(n^2)$$

算法的改进

原算法有4个子问题的递归求解，现在考虑：

$$wz + xy = (w + x)(y + z) - wy - xz$$

我们得到：

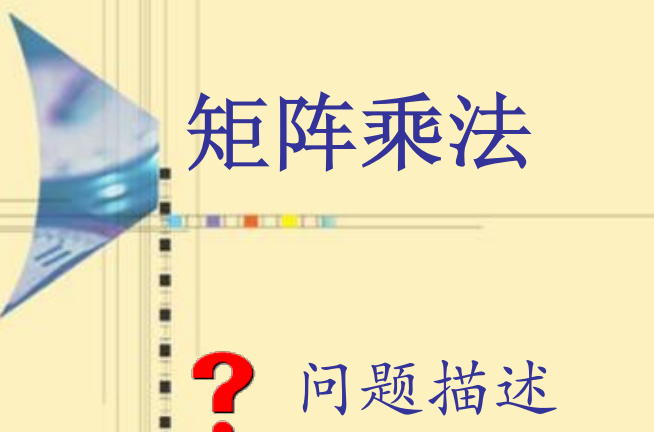
$$uv = wy2^n + ((w + x)(y + z) - wy - xz)2^{n/2} + xz$$

变为3个子问题的递归求解，因此时间复杂度是：

$$T(n) = \begin{cases} d & \text{if } n = 1 \\ 3T(n/2) + bn & \text{if } n > 1 \end{cases}$$

递推式的解是：

$$T(n) = \Theta(n^{\log_3}) = O(n^{1.59})$$



矩阵乘法

? 问题描述

- 令A和B是两个 $n \times n$ 的矩阵，我们希望计算它们的乘积 $C=AB$ 。



矩阵乘法

- 在传统方法中， $C=A \times B$ 是由以下公式计算的

$$C(i, j) = \sum_{k=1}^n A(i, k)B(k, j)$$

- 因为每计算一个元素 $C[i, j]$ ，需要做 n 个乘法和 $n-1$ 次加法
- 很容易看出算法需要 n^3 次乘法运算和 n^3-n^2 次加法运算。这导致其时间复杂度为 $\Theta(n^3)$ 。

? Is there much more efficient method?

矩阵乘法

假定 $n=2^k$, $k \geq 0$, 如果 $n \geq 2$, 则 A , B 和 C 可分别分成4个大小为 $n/2 \times n/2$ 的子矩阵

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

用分治方法来计算 C 的组成:

$$C = \begin{pmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{pmatrix}$$

矩阵乘法

计算量总耗费是：

$$T(n) = \begin{cases} m & \text{if } n = 1 \\ 8T(n/2) + 4(n/2)^2 a & \text{if } n \geq 2 \end{cases}$$

最终结果：

$$T(n) = (m + \frac{a2^2}{8-2^2})n^3 - (\frac{a2^2}{8-2^2})n^2 = mn^3 + an^3 - an^2$$

观察结论 3.4 分治方法需要 n^3 次乘法运算和 n^3-n^2 次加法运算。这些刚好和前面传统方法所讲述的一样。

- 两种算法的区别仅仅在于矩阵元素相乘的次序上，其他方面是一样的

算法改进-STRASSEN算法

基本思想

- 增加加减法的次数来减少乘法次数

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \quad \text{和} \quad B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$

$$C = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$

STRASSEN算法

我们首先计算以下的乘积矩阵

$$d_1 = (a_{11} + a_{22})(b_{11} + b_{22})$$

$$d_2 = (a_{21} + a_{22})b_{11}$$

$$d_3 = a_{11}(b_{12} - b_{22})$$

$$d_4 = a_{22}(b_{21} - b_{11})$$

$$d_5 = (a_{11} + a_{12})b_{22}$$

$$d_6 = (a_{21} - a_{11})(b_{11} + b_{12})$$

$$d_7 = (a_{12} - a_{22})(b_{21} + b_{22})$$

接着从一下面式子计算出C

$$C = \begin{pmatrix} d_1 + d_4 - d_5 + d_7 & d_3 + d_5 \\ d_2 + d_4 & d_1 + d_3 - d_2 + d_6 \end{pmatrix}$$

STRASSEN算法分析

算法用到的加法是18次，乘法是7次，对于其运行时间产生以下递推式

$$T(n) = \begin{cases} m & \text{if } n = 1 \\ 7T(n/2) + 18(n/2)^2 a & \text{if } n \geq 2 \end{cases}$$

或

$$T(n) = \begin{cases} m & \text{if } n = 1 \\ 7T(n/2) + (9a/2)n^2 & \text{if } n \geq 2 \end{cases}$$

$$T(n) = \left(m + \frac{(9a/2)2^2}{7-2^2}\right)n^{\log_7} - \left(\frac{(9a/2)2^2}{7-2^2}\right)n^2 = mn^{\log_7} + 6an^{\log_7} - 6an^2$$

即运行时间为：

$$\Theta(n^{\log_7}) = O(n^{2.81})$$

算法分析

- Strassen算法的高效之处，就在于，它成功的减少了乘法次数，将8次乘法，减少到7次。
- 不要小看这减少的一次，每递归计算一次，效率就可以提高 $1/8$ ，比如一个大的矩阵递归5次后， $(7/8)^5 = 0.5129$ ，效率提升一倍。
- 矩阵是稀疏矩阵时，为稀疏矩阵设计的方法更快。所以，稠密矩阵上的快速矩阵乘法实现一般采用Strassen算法。

三个矩阵乘算法的比较

三种算法所做的算术运算的次数

	乘法	加法	复杂度
传统算法.	n^3	$n^3 - n^2$	$\Theta(n^3)$
分治方法.	n^3	$n^3 - n^2$	$\Theta(n^3)$
STRASSEN算法.	$n^{\log 7}$	$6 n^{\log 7} - 6 n^2$	$\Theta(n^{\log 7})$

3.5.2 改进途径2-利用预处理减少递归内部计算量

- 航空气调度问题：设有 n 架飞机，为了安全调度，需要了解其中距离最近的两架飞机时什么飞，它们的距离是多少？

⇒ 平面最近点对问题



最近点对问题

? 问题描述:

- 设S是平面上n个点的集合，在S中找到一个点对p和q的，使其相互距离最短。
- 换句话说，希望在S中找到具有这样性质的两点 $p_1 = (x_1, y_1)$ 和 $p_2 = (x_2, y_2)$ ，使它们间的距离在所有S中点对间为最小。

$$d(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$



蛮力算法

- **蛮力算法**就是简单地检验所有可能的 $n(n-1)/2$ 个距离并返回最短间距的点对。耗费 $\Theta(n^2)$ 的时间
- 运用分治设计技术，并进行改进后，得到一个时间复杂度为 $\Theta(n \log n)$ 的算法来解决最近点对问题。

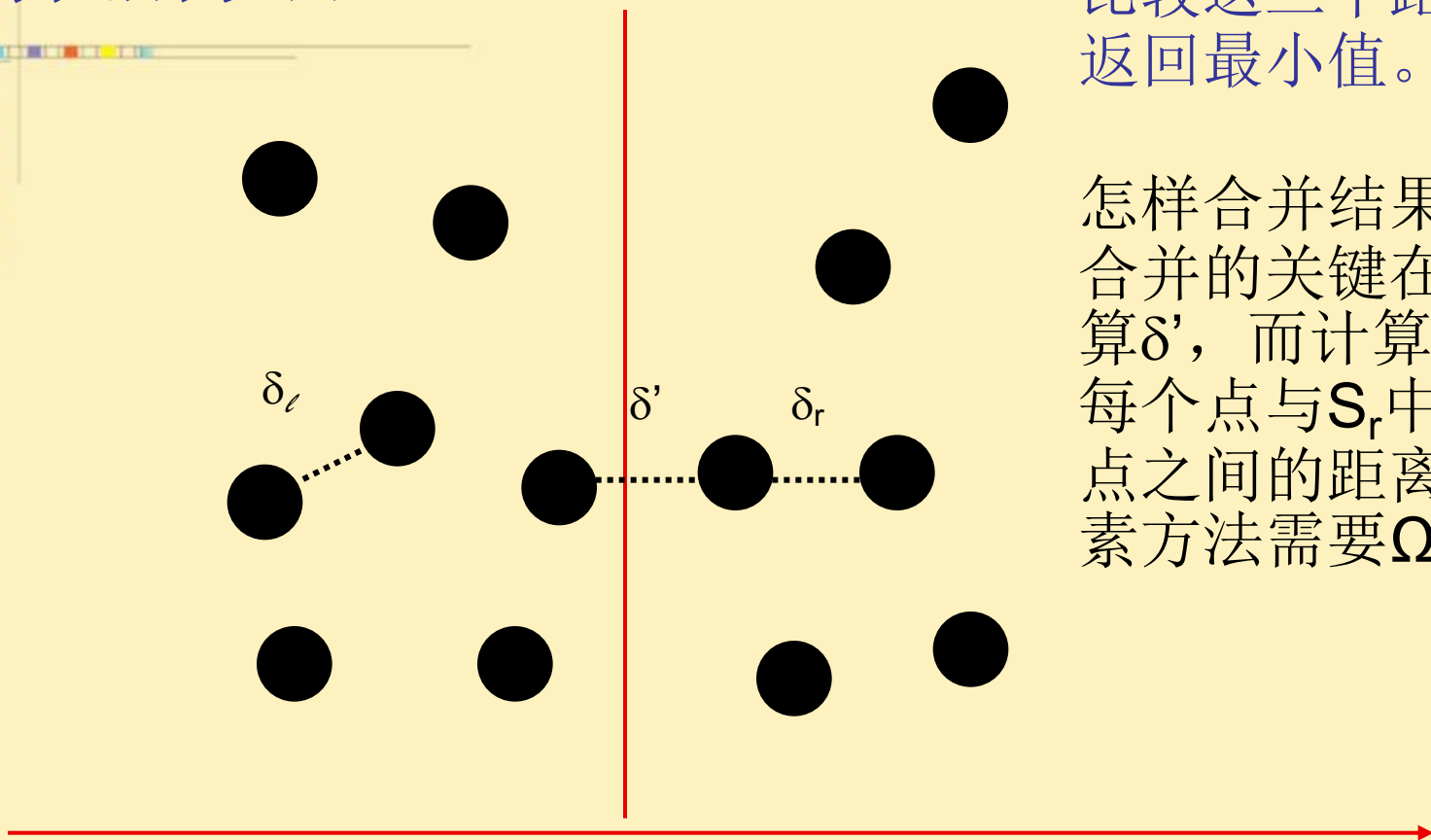
分治方法

- Sort:
 - 第一步是以x坐标增序对S中的点排序
- Divide:
 - S点集被垂直线L大约划分成两个子集 S_l 和 S_r ，使 $|S_l| = \lfloor |S|/2 \rfloor$ ， $|S_r| = \lceil |S|/2 \rceil$ ，设L是经过x坐标 $S[\lfloor n/2 \rfloor]$ 的垂直线，这样在 S_l 中的所有点都落在或靠近L的左边，而所有 S_r 中的点落在或靠近L的右边。
- Conquer:
 - 按上述递归地进行，两个子集 S_l 和 S_r 的最小间距 δ_l 和 δ_r 可分别计算出来。
- Combine:
 - 组合步骤是把 S_l 中的点与 S_r 中的点之间的最小间距 δ' 计算出来。最后所要求的解是 δ_l ， δ_r 和 δ' 中的最小值。

分治方法

比较这三个距离，
返回最小值。

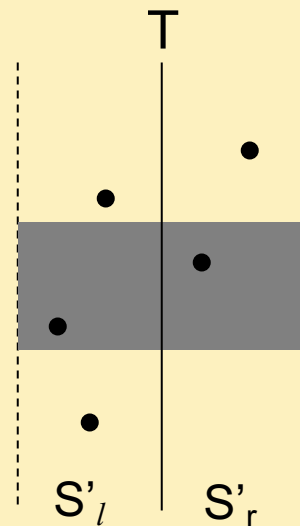
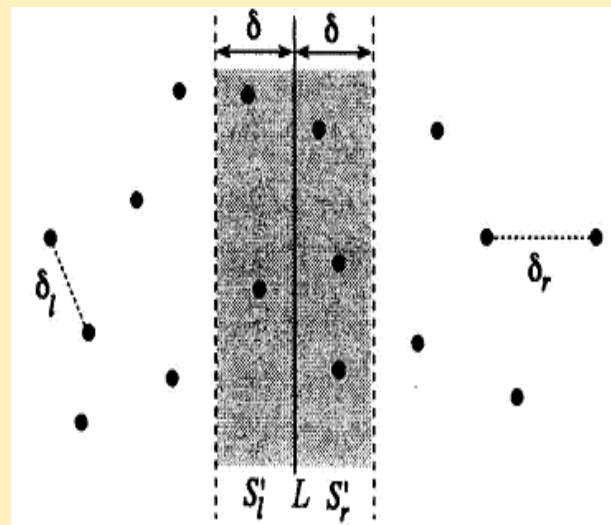
怎样合并结果？
合并的关键在于计算 δ' ，而计算 S_l 中的
每个点与 S_r 中每个
点之间的距离的朴素方法需要 $\Omega(n^2)$ 。



cdm

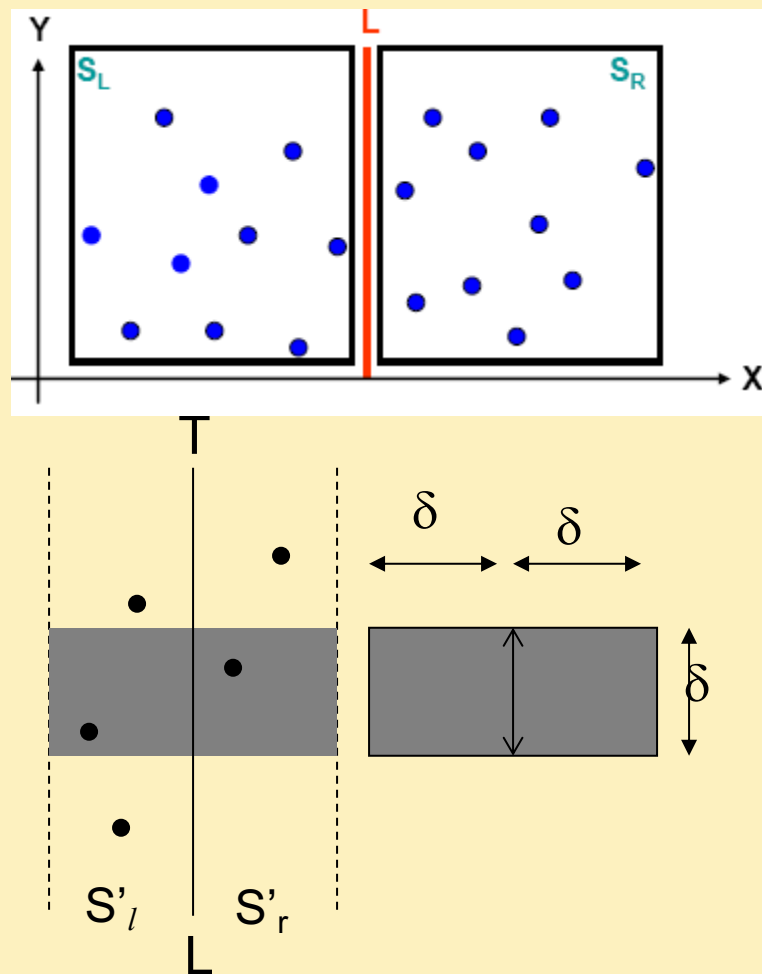
δ' 的计算

- 设 $\delta = \min\{\delta_l, \delta_r\}$
- 观察可知，如果最近点对由 S_l 中的某个点 p_l 与 S_r 中的某个点 p_r 组成，则 p_l 和 p_r 一定在划分线 L 的距离 δ 内。
- 因此，如果令 S'_l 和 S'_r 分别表示为在线 L 距离 δ 内的 S_l 和 S_r 中的点，则 p_l 一定在 S'_l 中， p_r 一定在 S'_r 中。
- 设 T 是两个垂直带内的点的集合，只 T 中点之间的最短距离才有可能小于 δ 。因此只需要计算 T 中点对之间距离。



δ' 的计算

- 假设 $\delta' \leq \delta$ ，则存在两点 $p_l \in S'_l$ 和 $p_r \in S'_r$ ，有 $d(p_l, p_r) = \delta'$ ，从而 p_l 和 p_r 之间的垂直距离不超过 δ
- 因此， p_l, p_r 这两点都在以垂直线 L 为中心的 $\delta \times 2\delta$ 矩形区内或其边界上。
- 因此，为了找到距离不超过 δ 的点对。对于一个点，可以通过设定矩形框，只需将它和矩形框中其他点计算距离即可。即对 T 中的每个点，以它为底边构造一个 $\delta \times 2\delta$ 的矩形，只需将它和矩形中的点求距离。（矩形框中有多少个点呢？）



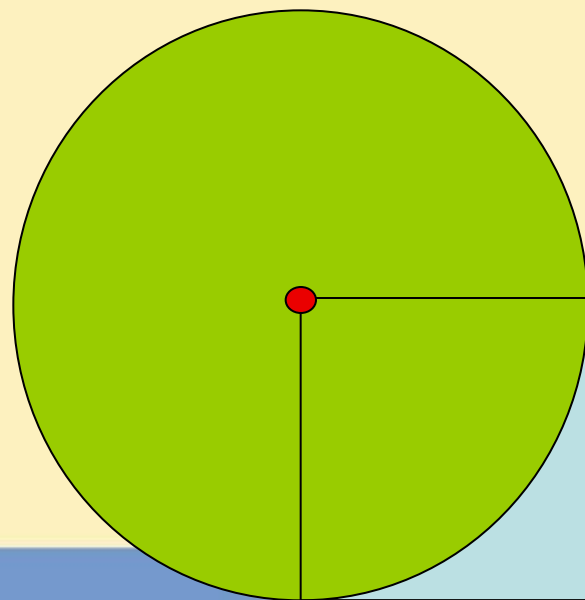
δ '的计算

定理 一个长为 δ 高为 δ 的矩形框中，要求任意两点之间的距离至少是 δ ，则该矩形框至多包含 4 个满足条件的点。

证明: 通过构造法进行证明。依次往矩形框中放置点，直到无法往矩形框加入点为止。

首先，想象一个围绕每个点的半径为 δ 的圆，代表我们不允许插入另一个点的区域。我们可以通过将点放置在矩形的角上来最小化这样一个圆与矩形的重叠面积，如右图所示。

因此，我们可以在浅蓝色区域内或圆的边缘放置点。

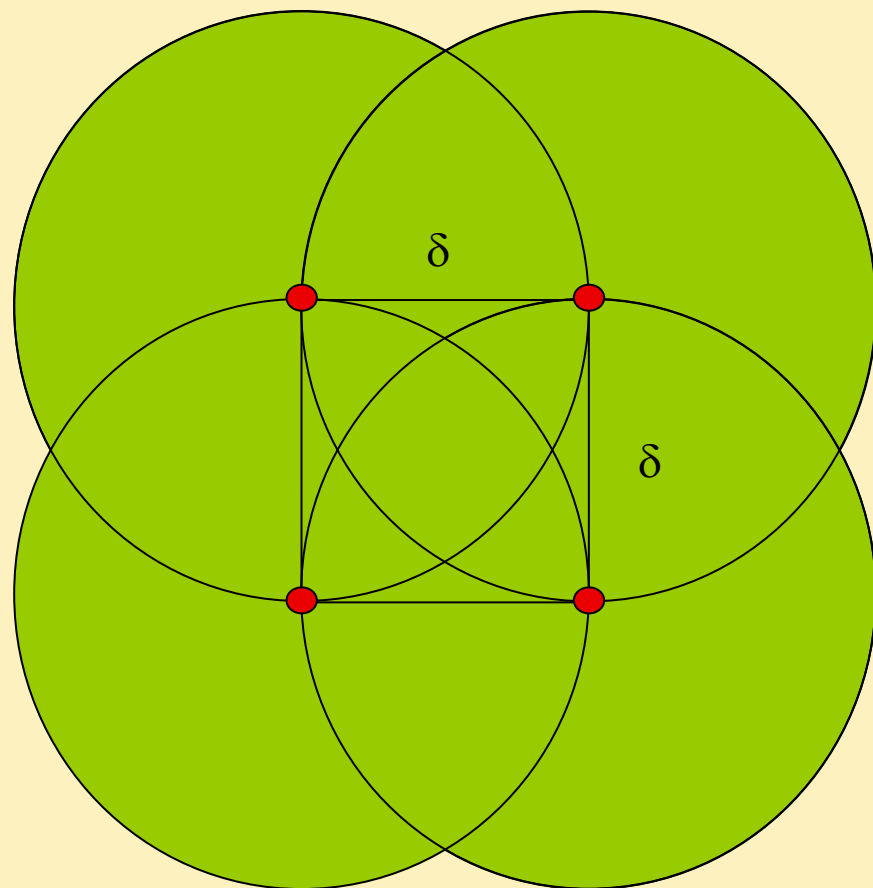


δ '的计算

接着，我们在矩形的右角再放一个点，画出以它为圆心的半径为 δ 圆。此时，除了剩下的两个角和底部的浅蓝色区域外，整个矩形都被覆盖了。。。

请注意，正方形中有4个点。如果试图在矩形的边界内沿任何方向移动这些点中的任何一个，那么将把两个点移动得太近了，及距离小于 δ 。

因此，如果不违反点之间的距离属性，我们就不可能在这个矩形中添加更多的点。因此，4是我们可以获得的最大点数。如果在 $\delta \times 2\delta$ 矩形区内，任意两点间的距离一定不低于 δ ，则这个矩形最多能容纳8个点，其中至多4个点属于 S_l ，4个点属于 S_r 。



δ' 的计算

观察结论 3.4

- T中的每个点最多需要和T中的7个点进行比较。
- 上述的观察结论仅给出与T中每个点p的比较点数的上界，而没有给出哪些点与p比较的任何信息。
- 稍想一下即可知，p一定是与T中临近的点比较。为了找到这样的相邻点，我们借助于以Y坐标的递增序对T中的点重新排序。然后，不难看出对T中每个点p，只需将它与Y坐标递增序下的7个点比较。

时间复杂性分析

- 排序S中的点需要 $\Theta(n \log n)$ 时间。
- 把点划分成 S_l 和 S_r 子集需要 $D(1)$ 时间，因为此时点已排序了。
- 至于组合步骤，我们可以看到它包含对T中点的排序和每点与其他至多7个点做比较，排序耗费 $O(n \log n)$ 时间，并存在至多 $7n$ 次比较，这样组合步骤最坏情况下需要 $\Theta(n \log n)$ 时间。
- 注意如果点数为3，则计算它们间最小间距只需要做三次比较，算法执行的递推关系变为：

$$T(n) = \begin{cases} 1 & \text{if } n = 2 \\ 3 & \text{if } n = 3 \\ 2T(n/2) + O(n \log n) & \text{if } n > 3 \end{cases} \quad \rightarrow T(n) = O(n \log^2 n)$$

算法的改进

增加预处理：将排序操作放到递归调用过程之前。

如果S中的元素按其y坐标一次性排序并存储在数组Y中，则每次我们需要在组合步骤中对T进行排序时，我们只需要在 $\Theta(n)$ 时间内从Y中提取其元素。这样递推式变为：

$$T(n) = \begin{cases} 1 & \text{if } n = 2 \\ 3 & \text{if } n = 3 \\ 2T(n/2) + \Theta(n) & \text{if } n > 3 \end{cases} \quad \rightarrow T(n) = O(n \log n)$$

算法3.8 CLOSESTPAIR

输入: 平面上 n 个点的集合 S 。

输出: S 中两点的最小距离。

1. 以 x 坐标增序对 S 中的点排序。
2. 序 $Y \leftarrow$ 以 y 坐标增序对 S 中的点排序。
3. $\delta \leftarrow cp(1, n)$.

过程 $cp(low, high)$

1. **if** $high - low + 1 \leq 3$ **then** 用直接方法计算 δ
2. **else**
3. $mid \leftarrow \lfloor (low + high) / 2 \rfloor$
4. $x_0 \leftarrow x(S[mid])$
5. $\delta_l \leftarrow cp(low, mid)$
6. $\delta_r \leftarrow cp(mid + 1, high)$
7. $\delta \leftarrow \min\{\delta_l, \delta_r\}$
8. $k \leftarrow 0$

To be continued...

过程 $cp(low, high)$

```
9.      for  $i \leftarrow 1$  to  $n$            {从Y中抽取T}
10.      if  $|x(Y[i]) - x_0| \leq \delta$  then
11.       $k \leftarrow k + 1$ 
12.       $T[k] \leftarrow Y[i]$ 
13.      end if
14.    end for
15.     $\delta' \leftarrow 2\delta$ 
16.    for  $i \leftarrow 1$  to  $k - 1$ 
17.      for  $j \leftarrow i + 1$  to  $\min\{i + 7, k\}$ 
18.        if  $d(T[i], T[j]) < \delta'$  then  $\delta' \leftarrow d(T[i], T[j])$ 
19.      end for
20.    end for
21.     $\delta \leftarrow \min\{\delta, \delta'\}$ 
22. end if
23. return  $\delta$ 
```

CLOSESTPAIR的时间复杂度分析

定理 3.7 假定有平面上 n 个点的集合 S , 算法CLOSESTPAIR在 $\Theta(n \log n)$ 时间内找到 S 中具有最小间距的一个点对。

分治法总结

- 常见的算法设计策略，很多问题应用；
- 最简单的二分，也可以是三分或更复杂的划分；
- 对输入数据的划分一般采用均衡原则，但分治算法归纳成的子问题不一定是对输入进行简单划分的结果；
- 无论什么方法归纳子问题，所得到的子问题必须与原问题类型一样，且子问题之间互相独立；
- 提高算法效率的方法一般是减少子问题个数和增加预处理以减少递归计算成本。